第 11 节 实操任务 1：实现 Reviewer Agent

> **学习目标**：review_node 能对知识条目进行四维度 LLM 评分，输出结构化 JSON 目标文件：`workflows/nodes.py`（review_node 正式版）

背景

V3 的核心是“反馈驱动的质量闭环”——Reviewer 评估，Reviser 修改，循环到通过。
本实操实现 Reviewer 节点。
核心原则：只评估不修改（Evaluate, don't modify）。

步骤 1：用 AI 编程工具生成 review_node 正式版

以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。
提示词：

```
请帮我编写 workflows/reviewer.py 中的 review_node 函数：

需求：
1. Reviewer 审核的对象是 state["analyses"]（不是 articles，articles 在 orgar
2. 5 维度评分，每维 1-10 分，权重如下：
   - summary_quality (摘要质量): 25%
   - technical_depth (技术深度): 25%
   - relevance (相关性): 20%
   - originality (原创性): 15%
   - formatting (格式规范): 15%
3. 用代码重算加权总分（不要信任模型算术）
4. 加权总分 >= 7.0 为通过
5. 只审核前 5 条 analyses（控 token 消耗）
6. temperature=0.1（评分一致性）
7. LLM 调用失败时自动通过（不阻塞流程）
8. 返回 {review_passed, review_feedback, iteration, cost_tracker}

依赖：
- chat_json(prompt, system=..., temperature=...) 返回 (parsed_json, usage
- accumulate_usage(tracker, usage)
- KBState 的 plan, analyses, iteration, cost_tracker 字段
```

生成的代码（参考实现）：

```
# 权重字典：写在代码里，方便不改 prompt 调整权重
```

```
REVIEWER_WEIGHTS = {
    "summary_quality": 0.25,
    "technical_depth": 0.25,
    "relevance":       0.20,
    "originality":     0.15,
    "formatting":      0.15,
}
REVIEWER_PASS_THRESHOLD = 7.0


def review_node(state: KBState) -> dict:
    """Reviewer 节点：对 analyses 进行 5 维度质量审核

    核心原则：只评估不修改（Evaluate, don't modify）。
    Reviewer 看到的是 Analyzer 输出的 analyses，不做任何改动，只给分 + 反馈。
    """
    analyses = state.get("analyses", [])
    iteration = state.get("iteration", 0)
    tracker = state.get("cost_tracker", {})

    if not analyses:
        return {
            "review_passed": True,
            "review_feedback": "没有条目需要审核",
            "iteration": iteration + 1,
        }

    # 只审核前 5 条，控制 token 消耗
    sample = analyses[:5]

    prompt = f"""你是知识库质量审核员。请审核以下分析结果：

{json.dumps(sample, ensure_ascii=False, indent=2)}

请按以下维度评分（每项 1-10 分）：
1. summary_quality  - 摘要质量
2. technical_depth  - 技术深度
3. relevance        - 相关性
4. originality      - 原创性
5. formatting       - 格式规范

请用 JSON 格式回复：
{{
    "scores": {{
        "summary_quality": 8,
        "technical_depth": 6,
        "relevance": 9,
        "originality": 5,
```

```
        "formatting": 8
    }},
    "feedback": "具体的改进建议（指出弱项）",
    "weak_dimensions": ["technical_depth", "originality"]
}}

当前是第 {iteration + 1} 次审核。"""

    try:
        result, usage = chat_json(
            prompt,
            system="你是严格但公正的知识库质量审核员。给出具体、可操作的反馈。",
            temperature=0.1,  # 低温度保证评分一致性
        )
        tracker = accumulate_usage(tracker, usage)

        # 【关键设计】用代码重算加权总分，不信任模型算术
        scores = result.get("scores", {})
        weighted_total = sum(
            scores.get(dim, 0) * w for dim, w in REVIEWER_WEIGHTS.items(
        )
        weighted_total = round(weighted_total, 2)
        passed = weighted_total >= REVIEWER_PASS_THRESHOLD

        feedback = result.get("feedback", "")
        weak_dims = result.get("weak_dimensions", [])
        if weak_dims:
            feedback = f"[弱项: {', '.join(weak_dims)}] {feedback}"

        print(
            f"[Reviewer] 加权总分: {weighted_total}/10, "
            f"通过: {passed} (第 {iteration + 1} 次审核)"
        )

    except Exception as e:
        passed = True
        feedback = f"审核 LLM 调用失败: {e}，自动通过"
        print(f"[Reviewer] 审核失败，自动通过: {e}")

    return {
        "review_passed": passed,
        "review_feedback": feedback,
        "iteration": iteration + 1,
        "cost_tracker": tracker,
    }
```

步骤 2：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释 review_node 的设计决策：`
> `1. 为什么 Reviewer 审 analyses 而不是 articles？`
> `2. 为什么权重字典 REVIEWER_WEIGHTS 写在代码里而不是 prompt 里？`
> `3. 为什么用代码重算加权总分，而不是让 LLM 自己算？`
> `4. 为什么 temperature=0.1？`
> `5. 为什么 LLM 调用失败时自动通过而不是报错？`

关键设计解读：

设计点	为什么这样做
只审核前 5 条	控制 token 消耗，避免长上下文降低审核质量
失败自动通过	审核是锦上添花，不能因为审核模块故障阻塞整个流水线
强制通过在 review 内	should_continue 只做简单路由判断，业务逻辑集中在节点内
temperature=0.2	审核需要高度一致性，多次审核同一内容应得到相近分数

步骤 3：确保 graph.py 使用 review_node

`workflows/graph.py` 应该在图里注册 review_node（v3 教师版已经配好）：

```
from workflows.reviewer import review_node

graph.add_node("review", review_node)
graph.add_edge("analyze", "review")  # 分析完后进入审核
```

步骤 4：运行验证

```
cd ~/ai-knowledge-base/v3-multi-agent
python3 -m workflows.graph
```

期望输出（standard 策略，max_iter=2）：

```
[Planner] 策略=standard, 每源=10 条, 阈值=0.5, 目标 10 条, 平衡模式
[Collector] 采集到 10 条原始数据
[Analyzer] 完成 10 条分析
[Reviewer] 加权总分: 7.2/10, 通过: True (第 1 次审核)
[Organizer] 整理出 10 条知识条目（准备入库）
[Organizer] 已写入 10 篇到磁盘
```

或者如果首次审核不通过（会看到下节实操 2 的 Reviser 介入）：

```
[Reviewer] 加权总分: 6.3/10, 通过: False (第 1 次审核)
[Reviser] 定向修改 5 条 analyses (迭代 1)
[Reviewer] 加权总分: 7.5/10, 通过: True (第 2 次审核)
[Organizer] 整理出 5 条知识条目（准备入库）
```

验证清单：

检查项	期望	实际
审核输出包含 overall_score	是	
审核输出包含 feedback	是	
review_passed 是 bool	是	
iteration 正确递增	是	

步骤 5：提交到 Git

```
git add workflows/reviewer.py
git commit -m "feat: implement review_node with 5-dim weighted scoring"
```

完成！ Reviewer 现在能对 analyses 做 5 维度加权评分，给出结构化反馈。下节实操 2 实现 Reviser，两者组成完整的审核修正循环。